

LINKSFOUNDATION.COM



Introduction to Pytorch Lightning

Structure, simplicity, scalability

LUCA BARCO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

What I need to code for a complete training loop?



Read and load data

**Separate
train/val/test splits**

Create batch



What I need to code for a complete training loop?



Dataset

Read and load data

**Separate
train/val/test splits**

Create batch



Model

**Model's
implementation**

**Explicit devices
management**

What I need to code for a complete training loop?



Dataset

Read and load data

**Separate
train/val/test splits**

Create batch



Model

**Model's
implementation**

**Explicit devices
management**



Evaluation

Loss Functions

Test Metrics

What I need to code for a complete training loop?



Dataset

Read and load data

**Separate
train/val/test splits**

Create batch



Model

**Model's
implementation**

**Explicit devices
management**



Evaluation

Loss Functions

Test Metrics



Logging

**Create graphs for
loss and test
metrics monitoring**

PyTorch

- The most **common** library for ML
- Full **control** of your workflow

But...

- Every functionality must be done **manually**
- **Hard** to **scale** to complex projects
- **Difficult** to keep **code clean and organized**
- Most of the time isn't spent designing models, **but rewriting training loops, evaluation, logging... again and again**



What is Pytorch Lightning?

- **PyTorch Lightning = PyTorch + Structure**
- Main objective: separate science from engineering
- **No need to rewrite** — focus on model design & tasks
- You just **set parameters** and **define** your ML **pipeline**
- Modular by design
 - Model → *LightningModule*
 - Data → *LightningDataModule*
 - Training loop → *Trainer*
 - Logging → *Logger / Callbacks*



Pytorch vs PyTorch Lightning

Pytorch: model + dataset + training loop
all in one script

```
train_loader = DataLoader(...)
val_loader = DataLoader(...)

for epoch in range(epochs):
    for x, y in train_loader:

        optimizer.zero_grad()
        loss = model(x).loss(y)
        loss.backward()
        optimizer.step()

    # validation loop...

    # checkpointing...
```

Pytorch vs PyTorch Lightning

Pytorch: model + dataset + training loop
all in one script

```
train_loader = DataLoader(...)
val_loader = DataLoader(...)

for epoch in range(epochs):
    for x, y in train_loader:
        optimizer.zero_grad()
        loss = model(x).loss(y)
        loss.backward()
        optimizer.step()

    # validation loop...

    # checkpointing...
```

Lightning: model (*LightningModule*) +
data (*LightningDataModule*) + *Trainer*

```
model = LitModel()
```

Pytorch vs PyTorch Lightning

Pytorch: model + dataset + training loop
all in one script

```
train_loader = DataLoader(...)
val_loader = DataLoader(...)

for epoch in range(epochs):
    for x, y in train_loader:

        optimizer.zero_grad()
        loss = model(x).loss(y)
        loss.backward()
        optimizer.step()

    # validation loop...

    # checkpointing...
```

Lightning: model (*LightningModule*) +
data (*LightningDataModule*) + *Trainer*

```
model = LitModel()
```

```
data = LitDataModule()
```

Pytorch vs PyTorch Lightning

Pytorch: model + dataset + training loop
all in one script

```
train_loader = DataLoader(...)
val_loader = DataLoader(...)

for epoch in range(epochs):
    for x, y in train_loader:
        optimizer.zero_grad()
        loss = model(x).loss(y)
        loss.backward()
        optimizer.step()

    # validation loop...

    # checkpointing...
```

Lightning: model (*LightningModule*) +
data (*LightningDataModule*) + *Trainer*

```
model = LitModel()
```

```
data = LitDataModule()
```

```
trainer = Trainer(max_epochs=10)
```

```
trainer.fit(model, data)
```

Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more powerful
- Encapsulates **model** + **training** logic

```
class LitClassifier(pl.LightningModule):  
    def __init__(self):  
        super().__init__()  
        self.model = nn.Linear(28*28, 10)
```

Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more **powerful**
- Encapsulates **model** + **training** logic

```
class LitClassifier(pl.LightningModule):  
    def __init__(self):  
        super().__init__()  
        self.model = nn.Linear(28*28, 10)
```

The model can be defined:

- inside the LightningModule
- outside and passed in (recommended for reusable modules across different models)

```
class LitClassifier(pl.LightningModule):  
    def __init__(self, model: nn.Module):  
        super().__init__()  
        self.model = model
```



Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more powerful
- Encapsulates **model** + **training** logic
- Key methods
 - **forward()**: model forward + other steps if needed

```
class LitClassifier(pl.LightningModule):  
    def __init__(self):  
        super().__init__()  
        self.model = nn.Linear(28*28, 10)  
  
    def forward(self, x):  
        return self.model(x)
```

Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more powerful
- Encapsulates **model** + **training** logic
- Key methods
 - `forward()`: model forward + other steps if needed
 - `training_step(batch, batch_idx)`: forward + training loss

```
class LitClassifier(pl.LightningModule):
    def __init__(self):
        super().__init__()
        self.model = nn.Linear(28*28, 10)

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = F.cross_entropy(y_hat, y)
        self.log("train_loss", loss)
        return loss
```

Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more powerful
- Encapsulates **model** + **training** logic
- Key methods
 - `forward()`: model forward + other steps if needed
 - `training_step(batch, batch_idx)`: forward + training loss
 - `validation_step(batch, batch_idx)`: forward + validation metrics
 - `test_step(batch, batch_idx)`: forward + test metrics

```
class LitClassifier(pl.LightningModule):
    def __init__(self):
        super().__init__()
        self.model = nn.Linear(28*28, 10)

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = F.cross_entropy(y_hat, y)
        self.log("train_loss", loss)
        return loss

    def validation_step(self, batch, batch_idx):
        ...

    def test_step(self, batch, batch_idx):
        ...
```

Lightning Module

your model + all training logic in one clean class.

- Similar to *torch.nn.Module* but more powerful
- Encapsulates **model** + **training** logic
- Key methods
 - `forward()`: model forward + other steps if needed
 - `training_step(batch, batch_idx)`: forward + training loss
 - `validation_step(batch, batch_idx)`: forward + validation metrics
 - `test_step(batch, batch_idx)`: forward + test metrics
 - `configure_optimizers()`: configure training optimizer + scheduler

```
class LitClassifier(pl.LightningModule):
    def __init__(self):
        super().__init__()
        self.model = nn.Linear(28*28, 10)

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = F.cross_entropy(y_hat, y)
        self.log("train_loss", loss)
        return loss

    def validation_step(self, batch, batch_idx):
        ...

    def test_step(self, batch, batch_idx):
        ...

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=1e-3)
```

Lightning Data Module

a reusable place for all your data logic.

- Define data loading logic

```
class DataModule(pl.LightningDataModule):  
    def __init__(self):  
        super().__init__()
```

Lightning Data Module

a reusable place for all your data logic.

- Define data loading logic
- Key methods
 - **prepare_data()**: download, preprocess

```
class DataModule(pl.LightningDataModule):  
    def __init__(self):  
        super().__init__()  
  
    def prepare_data(self):  
        ...
```

Lightning Data Module

a reusable place for all your data logic.

- Define data loading logic
- Key methods
 - `prepare_data()`: download, preprocess
 - `setup(stage)`: load data for the correct stage (train/val/test)

```
class DataModule(pl.LightningDataModule):  
    def __init__(self):  
        super().__init__()  
  
    def prepare_data(self):  
        ...  
  
    def setup(self):  
        ...
```

Lightning Data Module

a reusable place for all your data logic.

- Define data loading logic
- Key methods
 - `prepare_data()`: download, preprocess
 - `setup(stage)`: load data for the correct stage (train/val/test)
 - `train_data_loader()`
 - `val_data_loader()`
 - `test_data_loader()`:
instantiate torch DataLoader with training / validation/ test data

```
class DataModule(pl.LightningDataModule):
    def __init__(self):
        super().__init__()

    def prepare_data(self):
        ...

    def setup(self):
        ...

    def train_data_loader(self):
        ...

    def val_data_loader(self):
        ...

    def test_data_loader(self):
        ...
```

Lightning Data Module

a reusable place for all your data logic.

- Define data loading logic
- Key methods
 - `prepare_data()`: download, preprocess
 - `setup(stage)`: load data for the correct stage (train/val/test)
 - `train_data_loader()`
 - `val_data_loader()`
 - `test_data_loader()`: instantiate torch DataLoader with training / validation/ test data

```
class DataModule(pl.LightningDataModule):  
    def __init__(self):  
        super().__init__()  
  
    def prepare_data(self):  
        ...  
  
    def setup(self):  
        ...  
  
    def train_data_loader(self):  
        ...  
  
    def val_data_loader(self):  
        ...  
  
    def test_data_loader(self):  
        ...
```



Design
reusable
DataModules

Callbacks

Hooks to execute code at specific times

Useful to manage a **custom operations** in your training loop.

Built in callbacks

- EarlyStopping
- ModelCheckpoint
- LearningRateMonitor

```
class MyCallback(pl.Callback):
    def __init__(self):
        super().__init__()

    def on_train_start(self, trainer, pl_module):
        # Called at the beginning of training
        ...

    def on_train_end(self, trainer, pl_module):
        # Called at the end of training
        ...

    def on_epoch_start(self, trainer, pl_module):
        # Called at the beginning of each epoch
        ...

    def on_epoch_end(self, trainer, pl_module):
        # Called at the end of each epoch
        ...

    def on_train_batch_end(self, trainer, pl_module, outputs, batch, batch_idx):
        # Called at the end of every training batch
        ...
```

Callbacks

Hooks to execute code at specific times

Useful to manage a **custom operations** in your training loop.

Built in callbacks

- EarlyStopping
- ModelCheckpoint
- LearningRateMonitor

```
class MyCallback(pl.Callback):
    def __init__(self):
        super().__init__()

    def on_train_start(self, trainer, pl_module):
        # Called at the beginning of training
        ...

    def on_train_end(self, trainer, pl_module):
        # Called at the end of training
        ...

    def on_epoch_start(self, trainer, pl_module):
        # Called at the beginning of each epoch
        ...

    def on_epoch_end(self, trainer, pl_module):
        # Called at the end of each epoch
        ...

    def on_train_batch_end(self, trainer, pl_module, outputs, batch, batch_idx):
        # Called at the end of every training batch
        ...
```



Design
reusable
Callbacks

Loggers

- Support for several common loggers
- TensorBoardLogger (default)
- WandB, MLflow integrations

```
from pytorch_lightning.loggers import TensorBoardLogger,
CSVLogger, wandbLogger, MLFlowLogger

# TensorBoard logger (default folder: lightning_logs/)
tb_logger = TensorBoardLogger("logs/", name="my_experiment")

# CSV logger (saves metrics to CSV files)
csv_logger = CSVLogger("logs/", name="my_experiment")

# Weights & Biases logger
wandb_logger = wandbLogger(project="my_project_name",
name="experiment_1")

# MLflow logger
mlflow_logger = MLFlowLogger(experiment_name="my_experiment",
tracking_uri="file:./mlruns")
```

Loggers

- Support for several common loggers
- TensorBoardLogger (default)
- WandB, MLflow integrations



TensorBoardLogger
is the **standard** logger
if you don't specify one

```
from pytorch_lightning.loggers import TensorBoardLogger,
CSVLogger, wandbLogger, MLFlowLogger

# TensorBoard logger (default folder: lightning_logs/)
tb_logger = TensorBoardLogger("logs/", name="my_experiment")

# CSV logger (saves metrics to CSV files)
csv_logger = CSVLogger("logs/", name="my_experiment")

# Weights & Biases logger
wandb_logger = wandbLogger(project="my_project_name",
name="experiment_1")

# MLFlow logger
mlflow_logger = MLFlowLogger(experiment_name="my_experiment",
tracking_uri="file:./mlruns")
```

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling so you can **focus on your model and data.**

- **Single entryptpoint** for training and evaluation
- Handles:
 - **Gradients & devices**
 - **Training/validation/test loops**
 - **Callbacks** at the right time

Trainer

the **engine** that handles all training loops, logging, checkpointing, and scaling so you can **focus on your model and data**.

- **Single entrypoint** for training and evaluation
- Handles:
 - **Gradients & devices**
 - **Training/validation/test loops**
 - **Callbacks** at the right time

PyTorch

```
for batch in train_loader:  
    optimizer.zero_grad()  
    y_hat = model(batch)  
    loss = loss_fn(y_hat, batch.y)  
    loss.backward()  
    optimizer.step()
```

PyTorch Lightning

`trainer.fit()`

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling

```
trainer = pl.Trainer(  
    accelerator= "auto" ,      # "cpu", "gpu", "tpu", "mps", or "auto"  
    devices=1,                # number of devices (e.g. GPUs) to use  
    max_epochs=10,            # number of training epochs  
    precision= "16-mixed" ,    # optional: mixed precision training  
    log_every_n_steps=50,      # logging frequency  
    callbacks=[],             # list of callbacks (e.g. ModelCheckpoint, EarlyStopping)  
    logger=True                # logging (True = default TensorBoardLogger)  
)  
  
# Fit the model (training + validation)  
trainer.fit(model, datamodule=dm)  
  
# validate separately (optional)  
trainer.validate(model, datamodule=dm)  
  
# Test after training (optional)  
trainer.test(model, datamodule=dm)  
  
# Predict (optional)  
trainer.predict(model, datamodule=dm)
```

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling

Configuration
params for training
control, devices,
precision, etc.

```
trainer = pl.Trainer(  
    accelerator= "auto" ,      # "cpu", "gpu", "tpu", "mps", or "auto"  
    devices=1,                 # number of devices (e.g. GPUs) to use  
    max_epochs=10,             # number of training epochs  
    precision= "16-mixed" ,    # optional: mixed precision training  
    log_every_n_steps=50,      # logging frequency  
    callbacks=[],              # list of callbacks (e.g. ModelCheckpoint, EarlyStopping)  
    logger=True                 # logging (True = default TensorBoardLogger)  
)  
  
# Fit the model (training + validation)  
trainer.fit(model, datamodule=dm)  
  
# validate separately (optional)  
trainer.validate(model, datamodule=dm)  
  
# Test after training (optional)  
trainer.test(model, datamodule=dm)  
  
# Predict (optional)  
trainer.predict(model, datamodule=dm)
```

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling

Configuration
params for training
control, devices,
precision, etc.

Specify here your
callbacks

```
trainer = pl.Trainer(  
    accelerator= "auto" ,      # "cpu", "gpu", "tpu", "mps", or "auto"  
    devices=1,                # number of devices (e.g. GPUs) to use  
    max_epochs=10,            # number of training epochs  
    precision= "16-mixed" ,    # optional: mixed precision training  
    log_every_n_steps=50,      # logging frequency  
    callbacks=[],              # list of callbacks (e.g. ModelCheckpoint, EarlyStopping)  
    logger=True                # logging (True = default TensorBoardLogger)  
)  
  
# Fit the model (training + validation)  
trainer.fit(model, datamodule=dm)  
  
# validate separately (optional)  
trainer.validate(model, datamodule=dm)  
  
# Test after training (optional)  
trainer.test(model, datamodule=dm)  
  
# Predict (optional)  
trainer.predict(model, datamodule=dm)
```

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling

Configuration
params for training
control, devices,
precision, etc.

Specify here your
callbacks

Specify here your
logger

```
trainer = pl.Trainer(  
    accelerator= "auto" ,      # "cpu", "gpu", "tpu", "mps", or "auto"  
    devices=1,                 # number of devices (e.g. GPUs) to use  
    max_epochs=10,             # number of training epochs  
    precision= "16-mixed" ,    # optional: mixed precision training  
    log_every_n_steps=50,      # logging frequency  
    callbacks=[],              # list of callbacks (e.g. ModelCheckpoint, EarlyStopping)  
    logger=tb_logger           # logging (logger instance or True for TensorBoardLogger)  
)  
  
# Fit the model (training + validation)  
trainer.fit(model, datamodule=dm)  
  
# validate separately (optional)  
trainer.validate(model, datamodule=dm)  
  
# Test after training (optional)  
trainer.test(model, datamodule=dm)  
  
# Predict (optional)  
trainer.predict(model, datamodule=dm)
```

Trainer

the engine that handles all training loops, logging, checkpointing, and scaling

Configuration
params for training
control, devices,
precision, etc.

Specify here your
callbacks

Specify here your
logger

```
trainer = pl.Trainer(  
    accelerator= "auto" ,    # "cpu", "gpu", "tpu", "mps", or "auto"  
    devices=1,              # number of devices (e.g. GPUs) to use  
    max_epochs=10,          # number of training epochs  
    precision= "16-mixed" ,  # optional: mixed precision training  
    log_every_n_steps=50,    # logging frequency  
    callbacks=[],            # list of callbacks (e.g. ModelCheckpoint, EarlyStopping)  
    logger=tb_logger        # logging (logger instance or True for TensorBoardLogger)
```

```
# Fit the model (training + validation)  
trainer.fit(model, datamodule=dm)
```

training + validation

```
# validate separately (optional)  
trainer.validate(model, datamodule=dm)
```

validation

```
# Test after training (optional)  
trainer.test(model, datamodule=dm)
```

test

```
# Predict (optional)  
trainer.predict(model, datamodule=dm)
```

inference

Pytorch Lightning Best Practices

CODE STRUCTURE

- ✓ Keep **model** (LightningModule) and **data** (LightningDataModule) **separate**
- ✓ Use **forward()** only for **inference**
- ✓ Log metrics with **self.log()**
- ✓ Always return loss in **training_step**
- ✓ Centralize optimizers in **configure_optimizers()**

Pytorch Lightning Best Practices

CODE STRUCTURE

- ✓ Keep **model** (LightningModule) and **data** (LightningDataModule) **separate**
- ✓ Use **forward()** only for **inference**
- ✓ Log metrics with **self.log()**
- ✓ Always return loss in **training_step**
- ✓ Centralize optimizers in **configure_optimizers()**

TRAINER

- ✓ Debug fast with **fast_dev_run=True**
- ✓ Use **limit_train_batches** & **limit_val_batches** for quick tests
- ✓ Rely on **built-in callbacks**: ModelCheckpoint, EarlyStopping
- ✓ Ensure **reproducibility**: **Trainer(deterministic=True)** + seeds
- ✓ Extend with **callbacks**, not hacks in loops

Pytorch Lightning Best Practices

CODE STRUCTURE

- ✓ Keep model (LightningModule) and data (LightningDataModule) separate
- ✓ Use forward() only for inference
- ✓ Log metrics with self.log()
- ✓ Always return loss in training_step
- ✓ Centralize optimizers in configure_optimizers()

TRAINER

- ✓ Debug fast with **fast_dev_run=True**
- ✓ Use **limit_train_batches** & **limit_val_batches** for quick tests
- ✓ Rely on **built-in callbacks**: ModelCheckpoint, EarlyStopping
- ✓ Ensure **reproducibility**: **Trainer(deterministic=True)** + seeds
- ✓ Extend with **callbacks**, not hacks in loops

PERFORMANCE

- ✓ Start **small** (CPU) → **scale up** (GPU, multi-GPU) with Trainer flags
- ✓ Use **mixed precision** (precision=16) when possible
- ✓ Keep **preprocessing in DataModule**, not in the model
- ✓ **Profile with Trainer**(profiler="simple") or "advanced"
- ✓ Save hyperparams with **self.save_hyperparameters()**

Pytorch vs Lightning

Hands-on session